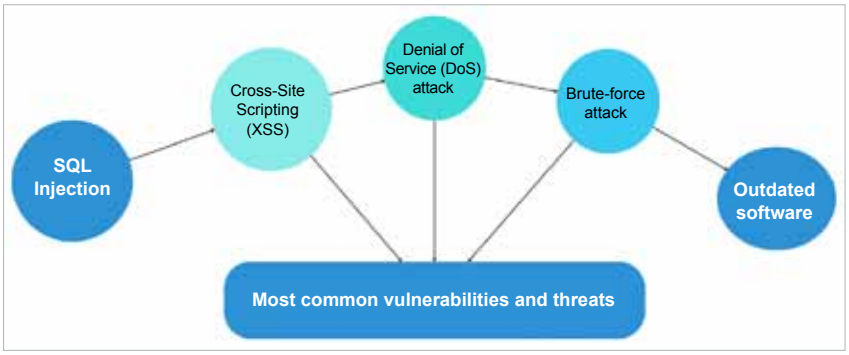

Figure 1: Various threats and vulnerabilities on web servers

- ***Legal implications:*** Data breaches can lead to legal issues, such as violations of data protection laws.

Tips on keeping a web server safe

To protect against threats, web server administrators should implement proactive security measures.

- Keep all software, including web server software and applications, up to date to patch known vulnerabilities.
- Use strong, unique passwords and consider implementing multi-factor authentication.
- Use firewalls to monitor and filter incoming and outgoing traffic, and IDS to detect and respond to suspicious activity.
- Follow security best practices, such as limiting access to sensitive data, encrypting data in transit and at rest, and regularly backing up data.

Best practices for securing web servers

Securing web servers is critical to protect your data and infrastructure from unauthorised access and cyberattacks. Here are the best practices for implementing authentication, authorisation, and encryption.

Authentication: Authentication is the process of verifying the identity of users or systems accessing a web server. It ensures that only authorised entities can access the server and its resources. Implement strong authentication mechanisms to verify the identity of users accessing your server such as:

- *Basic authentication:* Use HTTP basic authentication to prompt users for a user name and password before accessing the server. Here's an example using Apache's '.htaccess' file:

```
AuthType Basic
AuthName "Restricted Content"
AuthUserFile /path/to/.htpasswd
Require valid-user
```

Create the '.htpasswd' file with the 'htpasswd' utility:

```
htpasswd -c /path/to/.htpasswd username
```

- *Token-based authentication:* Implement token-based authentication for API access, where clients need to include a valid token in their requests. Use libraries like 'jsonwebtoken' in Node.js or 'PyJWT' in Python to generate and verify tokens.

```
const jwt = require('jsonwebtoken');

// Generate a token
const token = jwt.sign({ username:
'username' }, 'secretKey', { expiresIn:
'1h' });

// Verify a token
jwt.verify(token, 'secretKey', (err,
decoded) => {
  if (err) {
    console.error('Token is invalid');
  } else {
    console.log('Token is valid');
  }
});
```

Authorisation: Authorisation is the process of determining what actions or resources users or systems are allowed to access on a web server. It establishes permissions based on the authenticated identity of the user or system. Set up proper authorisation rules to control access to different parts of your web server. This can include:

- *Role-based access control (RBAC):* Assign roles to users and grant permissions based on those roles. For example, you can create roles like 'admin', 'user', and 'guest' with varying levels of access.

```
# Check if user has permission to access
a resource
def check_permission(user, resource):
   role = get_user_role(user)
   if role == "admin":
   return True  # Admin has full access
    elif role == "user" and resource ==
"restricted_resource":
        return True  # User has access
to specific resource
    else:
        return False  # Default deny
access
```

- *Directory-level access control:* Use directives in your web server configuration files (e.g., Apache's '<Directory>' directive) to restrict access to specific directories based on user roles.

```
<Directory /var/www/html/secure>
    Require role admin
</Directory>
```

Encryption: Encryption is the process of converting data into a secure format that can only be read or understood by authorised parties. It protects data in transit (e.g., over the network) and at rest (e.g., on disk) from unauthorised access